

Name: vaishnavi rajendra gayake

Subject:AIML

College name:T.b.girwalkar polytechnic college ambajogai

Branch:electronics and computer

Q1.# Create a class named Car

class Car:

```
def __init__(self, brand, model):
```

```
    self.brand = brand
```

```
    self.model = model
```

```
def display_info(self):
```

```
    print("Brand:", self.brand)
```

```
    print("Model:", self.model)
```

```
    print()
```

```
car1 = Car("Toyota", "Fortuner")
```

```
car2 = Car("Hyundai", "Creta")
```

```
car1.display_info()
```

```
car2.display_info() (Basic Class and Object)
```

Q2.(Using init Constructor)

class Book:

```
def __init__(self, title, author, price):
```

```
    self.title = title
```

```
    self.author = author
```

```
    self.price = price
```

```
def show_details(self):
```

```
    print("Title :", self.title)
```

```
    print("Author:", self.author)
```

```
    print("Price :", self.price)
```

```
print()

book1 = Book("Python Programming", "John Smith", 450)

book2 = Book("Data Structures", "Alice Brown", 550)

book1.show_details()

book2.show_details()
```

Q3.(Instance Methods and self)

```
class BankAccount:

    def __init__(self, account_holder, balance):

        self.account_holder = account_holder

        self.balance = balance

    def deposit(self, amount):

        self.balance += amount

        print(amount, "deposited successfully.")

    def withdraw(self, amount):

        if amount <= self.balance:

            self.balance -= amount

            print(amount, "withdrawn successfully.")

        else:

            print("Insufficient balance!")

    def show_balance(self):

        print("Current Balance:", self.balance)

account = BankAccount("Vaishnavi", 5000)

account.show
```

Q4.(Method with Parameters)

```
class Student:
```

```
    def __init__(self, name, marks):
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def calculate_grade(self):
```

```
        if self.marks >= 40:
```

```
            return "Pass"
```

```
        else:
```

```
            return "Fail"
```

```
    def show_details(self):
```

```
        print("Name :", self.name)
```

```
        print("Marks:", self.marks)
```

```
        print("Grade:", self.calculate_grade())
```

```
        print()
```

```
student1 = Student("Vaishnavi", 85)
```

```
student2 = Student("Rahul", 35)
```

```
student3 = Student("Sneha", 60)
```

```
student1.show_details()
```

```
student2.show_details()
```

```
student3.show_details()
```

Q5.(Multiple Methods)

```
class Employee:
```

```
    def __init__(self, name, salary):
```

```
        self.name = name
```

```
        self.salary = salary
```

```
    def display_details(self):
```

```
        print("Employee Name:", self.name)
```

```
        print("Salary:", self.salary)
```

```
    def give_raise(self, amount):
```

```
        self.salary += amount
```

```
        print("Salary increased by:", amount)
```

```
        print("New Salary:", self.salary)
```

```
    def yearly_bonus(self):
```

```
        return self.salary * 0.10
```

```
emp1 = Employee("Vaishnavi", 50000)
```

```
emp1.display_details()
```

```
emp1.give_raise(5000)
```

```
bonus = emp1.yearly_bonus()
```

```
print("Yearly Bonus:", bonus)
```

Q6.(Real-world Object Modeling).

```
class MobilePhone:
```

```
    def __init__(self, brand, model, battery_percentage):
```

```
        self.brand = brand
```

```
self.model = model

self.battery_percentage = battery_percentage

def charge(self, percent):

    self.battery_percentage += percent

    if self.battery_percentage > 100:

        self.battery_percentage = 100

    print(percent, "% charged.")

def use_phone(self, minutes):

    battery_used = minutes // 10

    self.battery_percentage -= battery_used

    if self.battery_percentage < 0:

        self.battery_percentage = 0

    print("Phone used for", minutes, "minutes.")

def show_status(self):

    print("\nBrand :", self.brand)

    print("Model :", self.model)

    print("Battery :", self.battery_percentage, "%")

phone = MobilePhone("Samsung", "Galaxy A55", 80)

phone.show_status()

phone.use_phone(30)

phone.show_status()

phone.charge(15)

phone.show_status()

phone.use_phone(100)

phone.show_status()
```

Q7.(Combining Concepts)

class Rectangle:

```
def __init__(self, length, width):
```

```
    self.length = length
```

```
    self.width = width
```

```
def area(self):
```

```
    return self.length * self.width
```

```
def perimeter(self):
```

```
    return 2 * (self.length + self.width)
```

```
def display(self):
```

```
    print("\nRectangle Details")
```

```
    print("Length   :", self.length)
```

```
    print("Width    :", self.width)
```

```
    print("Area     :", self.area())
```

```
    print("Perimeter :", self.perimeter())
```

```
length = float(input("Enter length: "))
```

```
width = float(input("Enter width: "))
```

```
rect = Rectangle(length, width)
```

```
rect.display()
```

Q8.(Updating Attributes)

class Player:

```
def __init__(self, name, score, level):
```

```
    self.name = name

    self.score = score

    self.level = level

def increase_score(self, points):

    self.score += points

    print(points, "points added.")

def level_up(self):

    self.level += 1

    print("Level Up!")

def show_progress(self):

    print("\nPlayer Progress")

    print("Name :", self.name)

    print("Score :", self.score)

    print("Level :", self.level)

player1 = Player("Vaishnavi", 100, 1)

player1.show_progress()

player1.increase_score(50)

player1.level_up()

player1.increase_score(100)

player1.level_up()

player1.increase_score(75)

player1.show_progress()
```


Q9.(Debugging OOP Code)

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def introduce(self):
```

```
        print("My name is", self.name, "and I am", self.age, "years old.")
```

```
p1 = Person("Vaishnavi", 25)
```

```
p1.introduce()
```

Q10.(Mini Project - Combined OOP)

```
class Book:
```

```
    def __init__(self, title, author):
```

```
        self.title = title
```

```
        self.author = author
```

```
        self.status = "Available"
```

```
    def issue_book(self):
```

```
        if self.status == "Available":
```

```
            self.status = "Issued"
```

```
            print(f'{self.title}' has been issued successfully.")
```

```
        else:
```

```
            print(f'{self.title}' is already issued.")
```

```
    def return_book(self):
```

```

        if self.status == "Issued":

            self.status = "Available"

            print(f"'{self.title}' has been returned successfully.")

        else:

            print(f"'{self.title}' is already available.")

def show_info(self):

    print(f"Title : {self.title}")

    print(f"Author: {self.author}")

    print(f>Status: {self.status}")

    print("-" * 30)

library = []

while True:

    print("\n===== LIBRARY MANAGEMENT SYSTEM =====")

    print("1. Add New Book")

    print("2. Issue Book")

    print("3. Return Book")

    print("4. Show All Books")

    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":

        title = input("Enter Book Title: ")

        author = input("Enter Author Name: ")

        book = Book(title, author)

        library.append(book)

        print("Book added successfully!")

```

```
elif choice == "2":  
    title = input("Enter book title to issue: ")  
    found = False  
    for book in library:  
        if book.title.lower() == title.lower():  
            book.issue_book()  
            found = True  
            break  
    if not found:  
        print("Book not found!")  
elif choice == "3":  
    title = input("Enter book title to return: ")  
    found = False  
    for book in library:  
        if book.title.lower() == title.lower():  
            book.return_book()  
            found = True  
            break  
    if not found:  
        print("Book not found!")  
elif choice == "4":  
    if len(library) == 0:  
        print("No books available in library.")  
    else:  
        print("\n--- Library Books ---")
```

```
        for book in library:
            book.show_info()
elif choice == "5":
    print("Thank you for using Library Management System!")
    break
else:
    print("Invalid choice! Please try again.")
```